

EJBCA Workflow Demos

with Kubernetes Cert-Manager

- Illustrated **Pull-Request** validations, for two unrecognized garbage-collection gaps in **EJBCA** CE & EE.
- Simple scripted mechanism to **deploy** EJBCA-CE in Docker containers, then adding the Pull Requests.
- Automated K8s certificate requests, with a PKI tool implementing the **Keyfactor** tutorial steps (**DEK**).
- Certificate lists by EJBCA end-entity, clearly showing: Active, Expired, Revoked (**ELT** PKI tool).
- User-friendly certificate content inspection (**cert-grep** & **ssl-grep** PKI tools).
- Automated and well-documented **workflow** examples, for easy reproduction and demonstrations.
- **AI-assisted** development, with LLM notes (Claude files) facilitates uptake.
- Open-source distribution on **GitHub**, of all notes and tools.

by **JohnB**, with AI pair-programming using Anthropic Claude Code (Opus) · June 23rd, 2026

AI assist

- ✓ **Claude Code** PowerPoint and MD (markdown) templating, with document suggestions.
- ✓ Substantial Keyfactor GitHub source-code review and further developments by **Claude Code** (Opus 4.x).
- ✓ All code & config artifacts were directed and reviewed by **JohnB**, with full PKI DevOps & programming expertise.

Project Overview

- Two EJBCA-CE Pull Requests (PRs), verified from a clean clone:
<https://github.com/John-D-B/Claudes/blob/main/2026-06-01.EJBCA-tools/elt/insights/CLAUDE-ejbca-fix-requests.md>
 - Fix-26 — REST-API DELETE of one or more revoked certificate, when expiration date is far future.
 - Fix-27 — DBMS worker that bulk-deletes revoked certs (garbage collection, prior to EJBCA-Issuer fix).
- By-products demonstrating the PRs, but also useful stand-alone:
 - Scripted (automated) local setup of EJBCA-CE in Docker, directly from GitHub source, for quick onboarding.
 - Python “EJBCA Lifecycle Tool” (**ELT**) showing organized lists of the X.509 certificates within EJBCA.
 - Scripted (automated) K8s cert-manager issuing tool (**DEK**), based on Keyfactor tutorial of manual steps:
<https://docs.keyfactor.com/ejbca/9.3.2/tutorial-use-ejbca-with-cert-manager>
 - User-friendly, high-powered X.509 certificate inspection tools **cert-grep** & **ssl-grep**, resolving this gripe:
*“Everyone **talks** about PKI certificates, but no one **conveniently shows me** what you’re talking about !”*

AI assist

- ✓ Turned and tested two upstream PRs into a live demo.
- ✓ Refined and documented this into a working local setup.

- ✓ **Weeks & Months of know-how study reduced to Hours.**
- ✓ Tested and repeatable from a clean GitHub clone.

Problem description: Corporate scenario

- The PKI / EJBCA **team** is a separate silo from the Kubernetes / RHOS admin **teams**, which need certificates.
- K8s **cert-manager** is an obvious integration tool, BUT it appears to K8s teams as the **real** “CA/RA” server.
- Effectively, K8s cert-manager **is**, rightly, the “CA/RA” for the K8s team. (*Sorry Keyfactor, you’re just a backend.*)
- So: bugs in EJBCA $\leftrightarrow$ K8s integration are “**unsolvable**” to EJBCA team, and “**invisible**” to K8s/RHOS teams.
- Real example: PKI security team defines a “max certificate lifetime” of **two years**, or maybe **47 days**, in EJBCA.
- Then the K8s wise-guys / stress-testers / zero-trusters request certs with an **hourly renewal**. Because they can.
- In EJBCA, if “**Enforce unique DN**” is checked, the first renewal immediately fails – the CN (etc) is not unique.
- K8s bug fix: unchecking it in EJBCA resolves symptoms, but then “garbage certificates” **accumulate hourly!**
- Current EJBCA-EE (only) can be configured with a “DBMS reaper” service for **expired** certs, but not for **revoked**.
- Earlier EJBCA-Issuer v1.3.2 bug (now fixed by me) didn’t propagate “NotAfter” (expired) date when issuing.
- Real example: **Many thousands** of **garbage certs** accumulating hourly, with no recourse! Tech support no help.

AI assist

- ✓ EJBCA-EE Software Appliance has no tool for this.
- ✓ Claude did in-depth analysis via EJBCA REST-API.

- ✓ **Deduced that EJBCA organizes by End-Entity, not Cert.**
- ✓ Bugs and gaps detected and reported (in my docs).

Example

This small example shows one active cert (with the same CN), and a **dozens** of revoked certs that accumulate with **blind K8s automation**. This “revoked” status (“**R**”) is only because EJBCA “**Single Active Cert**” is checked, else they’d also be active – even worse! Certificates with status “**r**” are especially problematic, as they can’t be “reaped” by EJBCA until they expire. The purpose of Pull Requests **Fix-26** and **Fix-27** is to enable deleting “**r**” certs in EJBCA-EE, and both “**R**” and “**r**” in EJBCA-CE.

```
$ elt list4 -ee-username 'TEST default-duration-1h'
```

ejbca-lifecycle-tool.py v5.8.0

Certificates: 39: 1:A active, 0:E expired, 36:R revoked+expired, 2:r revoked+unexpired

#	S	Serial	Not Before	Not After
1	A	7f66daccfafd22d8d8cbd970a6bf5181884a1234	May 06 05:35:14 2026 GMT	May 06 06:45:14 2026 GMT
2	R	251730a8c86b713405370ffb7d7a330491cb5678	May 06 04:58:33 2026 GMT	May 06 06:08:32 2026 GMT
3	R	7c0d8bf1115c42367b0245b83aca75b61e091011	May 06 04:21:52 2026 GMT	May 06 05:31:52 2026 GMT
4	R	66e3dcd5892e39d628df4026bbb845a28d221213	May 06 03:45:11 2026 GMT	May 06 04:55:11 2026 GMT
5	r	775dbfe747a506e639f607293b86500b30ca1415	May 06 03:45:11 2026 GMT	Jun 10 04:55:11 2027 GMT
6	r	21d77603f83c570bd50f95e0d60e2dc9d6bd1517	May 06 03:08:28 2026 GMT	Jun 10 04:18:28 2027 GMT

... 33 more – use -F to see the full list

AI assist

- ✓ Write Python tool: **EJBCA Lifecycle Tool (ELT)**.
- ✓ Review EJBCA REST-API response and deduce status.
- ✓ Synthesize **WSDL** for EJBCA-CE, for missing REST-API's.
- ✓ Test and ensure EJBCA-CE & EJBCA-EE symmetry.

How to quickly reproduce this issue?

- Setting up and configuring a new EJBCA environment is non-trivial, as is integrating Kubernetes cert-manager – to say nothing about detecting bugs and fixing them.
- Even if you have good DevOps skills, there is much new and non-standard here.
- **JohnB's goal:** Create a demo ecosystem, to illustrate these PR's for a wider audience.
 - AI assistance makes this feasible, for a solo PKI practitioner with limited time.
Anthropic Claude Code is my preferred partner.
 - Deliverable: A comprehensive guide **and** software (PKI tools) to quickly implement these test cases. These are the missing integration pieces not provided by Keyfactor.
 - Published on GitHub:
<https://github.com/John-D-B/Claudes/blob/main/2026-06-01.EJBCA-tools/README.md>
<https://github.com/John-D-B/Claudes/blob/main/2026-06-01.EJBCA-tools/ejbca-ce/README-ejbca-ce.md>

AI assist

- ✓ Multiple Claude Code instances working in parallel.
- ✓ Each PKI tool and workflow is AI-assisted and reviewed.

✓ Documentation in markdown format, with skill guidelines.

Two ways to run it

- Automated: the orchestrator scripts run it for you
<https://github.com/John-D-B/Claudes/blob/main/2026-06-01.EJBCA-tools/ejbca-ce/DEMO-automated.md>
- Manual: a step-by-step run book — every command + expected output
<https://github.com/John-D-B/Claudes/blob/main/2026-06-01.EJBCA-tools/ejbca-ce/DEMO-manually.md>
- Both stay consistent, from one shared workflow description

AI assist

- ✓ Two compatible walkthroughs, one model.
- ✓ Automated + manual stay in lockstep.

- ✓ Every manual step shows expected output (log files).
- ✓ Section numbering aligned across both.

Workflow Summary: Automated

<https://github.com/John-D-B/Claudes/blob/main/2026-06-01.EJBCA-tools/ejbca-ce/DEMO-automated.md>

- Part 1: Bring up the Docker stack
- Part 2: One-time setup (script group: 210.bootstrap/)
- Part 3: Build the PR code into the stack (230.rebuild/)
- Part 4: Verify both PRs (500.verify-PR/)
- Part 5: The real-world finale (BYOC mode)
 - B.Y.O.C. = *“Bring Your Own Certificates”*

AI assist

- ✓ Bash scripts written to automate this.
- ✓ Scanned scripts + docs for inconsistencies.

- ✓ Nothing secret written into the clone.
- ✓ Reconciled results in a single pass.

Workflow Summary: Manually

<https://github.com/John-D-B/Claudes/blob/main/2026-06-01.EJBCA-tools/ejbca-ce/DEMO-manually.md>

- A. Set up the ecosystem and tools
- B. Build the EJBCA-CE server (Docker)
- C. Build the k3d cluster for K8s cert-manager (Docker)
- D. Build up the revoked-certificate accumulation (DEK)
- E. Apply the two PR fixes (visible Reaper results on the D01 accumulation)
- F. Show the results (ELT, cert-grep, ssl-grep)

Summary: How all this was built

- Development environment: JohnB's **Mac** M2 Laptop, with Bash and “Homebrew” addons. All scripts and workflow are also expected to work on any Linux system, and in WSL.
- Several AI code-bots used in parallel, coordinated via **Rchat** - a custom AI communication bus.
- Scripted setup of EJBCA-CE, pulling from GitHub, building into a fully functional app.
- Custom tools (ELT, DEK, cert-grep, ssl-grep) built with Claude Code assistance.
- Tested: per-PR integration tests + fully captured run
- Several refactoring passes, goals carried in parallel

AI assist

- ✓ Multiple code-bots, coordinated via Rchat
- ✓ Custom tooling built with Claude Code

- ✓ Codebase reading + cross-checking throughout
- ✓ Tested with integration + full-run capture

Bonus · another PKI tool

cert-grep & ssl-grep: X.509 Certificate Inspectors

cert-grep reads certificate files. **ssl-grep** reads live endpoints. Same plain output, based on “nice-ified” OpenSSL.

- <https://gitlab.com/umi-ch/cert-grep>

```
$ cert-grep.py ManagementCA.crt summary_0 # - Show a summary.
0: Certificate:
  Issuer:      CN = ManagementCA
  Subject:     CN = ManagementCA
  Serial Number: 63:66:d2:a8:46:1b:b3:42:2f:2b:2a:46:6e:5e:be:e6:4e:8d:5b:94
  Not Before:   Jun 21 22:13:13 2026 GMT
  Not After:    Jun 20 22:13:12 2036 GMT
```

```
$ ssl-grep.py https://host.k3d.internal:8443 summary_0 verify # - Also verify the chain.
# Chain verification: 2 certificate(s)
#   - chain: [0] host.k3d.internal ← leaf
#           + Valid (expires Jun 20 22:05:14 2028 GMT)
#           + Signed by ManagementCA
# Chain verification result: chain is internally consistent
```

AI assist

- ✓ **cert-grep + ssl-grep** are built with Claude assistance.
- ✓ Chain verify & weakness scans are built in.

- ✓ One engine: Files, P12/JKS, live SSL/TLS.
- ✓ Readable - without OpenSSL incantations.

10 / 10